

SMART CLINIC SYSTEM

Architecture and Prototype Report

Analysis • Architectural Drivers • DDD • Microservices • Event-Driven Workflows • Validation • Deployment

Java 17 | Spring Boot | RabbitMQ | PostgreSQL | Stripe | Docker

3 August 2026

Academic prototype — implemented behavior is distinguished from proposed production improvements.

Executive Summary

The Smart Clinic System is a distributed prototype for appointment scheduling and financial processing. Scheduling owns doctor calendars, appointment lifecycle rules, temporal validation, and conflict resolution. Billing owns pricing profiles, proforma invoices, payment and refund ledgers, policy evaluation, timeout processing, and the Stripe integration. The two services own separate databases and exchange lifecycle facts through RabbitMQ.

The implementation is a strong match for the Software Architecture and Platforms course because architectural decisions are visible in both design and code: Domain-Driven Design defines the model and boundaries; Hexagonal Architecture isolates infrastructure; the microservice split establishes independent deployment and data ownership; asynchronous messaging creates temporal decoupling and eventual consistency; and Docker Compose provides a reproducible deployment.

This revision organizes the project from analysis to deployment, as requested by the course exam guidance. It adds explicit functional and non-functional requirements, architectural drivers, Quality Attribute Scenarios, user stories, a context map, three architecture views, ADRs, testing-pyramid analysis, validation evidence, proposed SLOs/SLIs, and a transparent pattern-selection discussion. [SAP-EXAM, SAP-QUESTIONS, SAP-A1]

Submission position

The project is not presented as a copy of the course assignments. It is a student-proposed system that applies the course concepts relevant to its business problem. Patterns that are not justified by the prototype—such as Event Sourcing, an API Gateway, Kafka, and Kubernetes—are explicitly discussed rather than added only to satisfy a checklist.

Report Map

1. Problem analysis: stakeholders, user stories, Event Storming, language, and invariants.
2. Requirements and architectural drivers: FRs, NFRs, ASRs, Quality Attribute Scenarios, and traceability.
3. Domain and architecture design: bounded contexts, context map, aggregates, events, and architectural views.
4. Patterns and decisions: Hexagonal Architecture, microservices, messaging, CQRS, ADRs, and excluded patterns.
5. Implementation, validation, deployment, demonstration, limitations, and course-alignment appendices.

1. Problem Analysis

1.1 Domain and Motivation

A clinic appointment is more than a calendar row. It is constrained by a doctor's effective working schedule, temporary unavailability, existing active bookings, fixed duration, time alignment, appointment state, payment deadline, cancellation initiator, and refund policy. These rules create two related but distinct decision areas: operational scheduling and financial settlement.

The prototype separates these areas because they change for different reasons. A new scheduling rule should not modify payment-provider code. A new refund policy should not modify calendar conflict detection. The same appointment identifier crosses the boundary, but each context protects its own model and persistence.

1.2 Stakeholders

Stakeholder	Architecturally relevant concern
Patient	Books, pays for, reschedules, and cancels appointments; expects clear financial consequences.
Doctor	Publishes working schedules and unavailability; needs conflict-free appointments.
Clinic reception	Operates scheduling lifecycle actions and may cancel appointments on behalf of the clinic.
Billing/finance staff	Needs invoice, payment, refund, and reconciliation visibility.
System operator	Deploys the services and monitors RabbitMQ, PostgreSQL, and application behavior.
Stripe	External payment provider used for PaymentIntent and refund processing; confirms outcomes by signed webhook.

1.3 Core Business Capabilities

Capability	Responsibility
Scheduling	Book, cancel, check in, complete, mark no-show, and reschedule appointments.
Calendar management	Maintain effective schedules and unavailability; identify invalidated future bookings.
Conflict detection	Reject bookings outside working time or overlapping unavailability/active appointments.
Pricing and invoicing	Create a proforma, evaluate discounts and tax, and lock the checkout amount.
Payment and refund	Initiate Stripe operations, store ledger records, and finalize from signed webhooks.
Timeout processing	Release a checkout window or expire an unpaid invoice and cancel its appointment.
Queries	Expose purpose-specific appointment, invoice, and payment projections.

1.4 User Stories and Acceptance Criteria

US-01 — As a patient, I want to book an available one-hour appointment so that I can reserve a consultation.

Acceptance criteria: The request must fit an effective working shift, use a 15-minute boundary, and not overlap unavailability or another active booking. A successful booking emits AppointmentWasBooked.

US-02 — As a patient or receptionist, I want to cancel an unpaid appointment so that the time slot is released without contacting the payment provider.

Acceptance criteria: Scheduling marks the appointment cancelled and emits AppointmentWasCanceled. Billing moves a DRAFT invoice to CANCELLED and emits no refund request.

US-03 — As a patient or receptionist, I want a paid cancellation to be evaluated under the clinic refund policy.

Acceptance criteria: Billing evaluates initiator, time since booking, and notice before the appointment. Eligible cancellations enter REFUND_PENDING and are finalized only after a signed Stripe webhook.

US-04 — As the clinic, I want abandoned invoices to expire automatically so that unpaid appointments do not remain reserved indefinitely.

Acceptance criteria: Billing scans invoices on a schedule, publishes InvoiceWasExpired, and Scheduling cancels the appointment with SYSTEM_AUTOMATION.

US-05 — As a doctor or receptionist, I want schedule changes and unavailability to invalidate conflicting future bookings.

Acceptance criteria: The calendar domain detects affected appointments, cancels them with SYSTEM_AUTOMATION, and publishes cancellation events.

Example acceptance scenario — conflicting booking

Given a doctor already has an active appointment from 10:00 to 11:00, when another booking requests 10:30 to 11:30, then the request is rejected, no appointment is persisted, and no AppointmentWasBooked event is published. This scenario is enforced by AppointmentCalendar and AppointmentConflictDomainService.

1.5 Event Storming View

The main workflow can be expressed as commands, aggregates, events, and policies. Orange items are facts that have already happened; purple items are policies or consumers that react to those facts. This model links analysis terminology directly to the implemented classes.

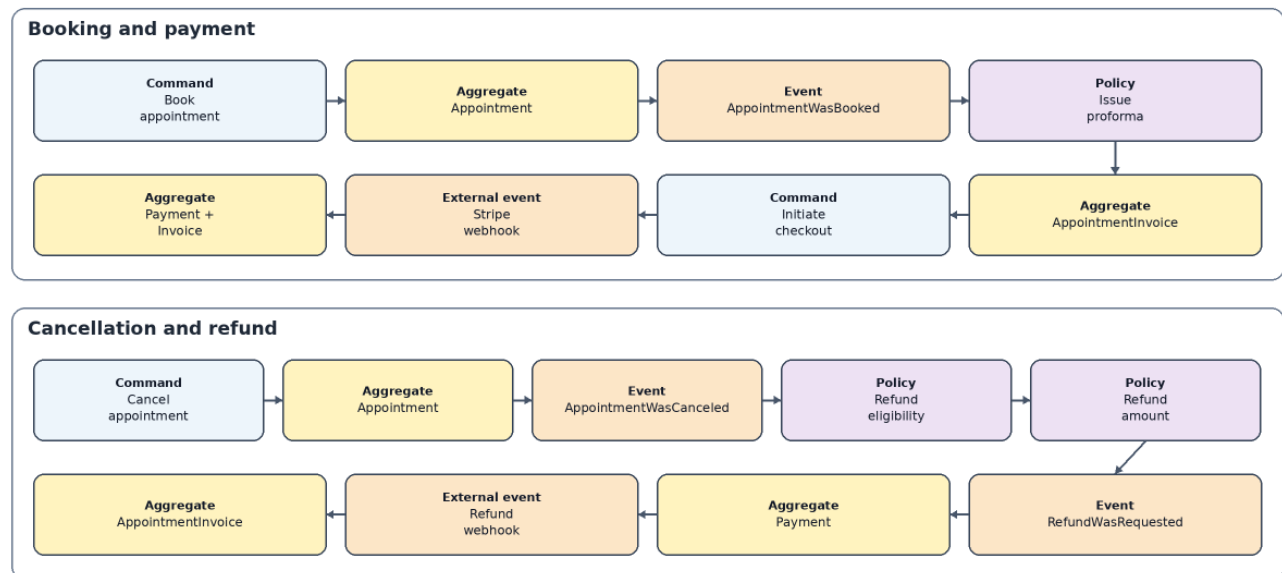


Figure 1. Event Storming-style view of booking/payment and cancellation/refund workflows.

1.6 Ubiquitous Language

Term	Meaning in this project
Appointment	A one-hour reservation between one doctor and one patient.
Active appointment	An appointment in SCHEDULED or CHECKED_IN state.
Appointment Calendar	Doctor-specific aggregate that owns effective schedules and unavailability.

Term	Meaning in this project
Effective Schedule	A weekly schedule valid from a specific date; the latest applicable schedule is selected.
Sharp Time	A time aligned to 15-minute increments with no seconds or nanoseconds.
Working Shift	A start/end interval inside which an appointment must fit completely.
Unavailability	Temporary period during which the doctor cannot accept appointments.
Proforma Invoice	Draft financial document created after booking and repriced before checkout.
Payment window	Temporary checkout interval during which the amount is locked.
Payment ledger	Payment aggregate entry for a deduction or refund and its gateway transaction identifier.
Cancellation Initiator	PATIENT, CLINIC_RECEPTION, or SYSTEM_AUTOMATION.
Refund eligibility	Decision whether a paid cancellation may produce a refund.
Refund strategy	Rule that calculates the approved refund amount.
Eventual consistency	Temporary divergence between Scheduling and Billing while events are in transit.
Published Language	The integration-event contracts shared between bounded contexts.

1.7 Core Domain Invariants

Scheduling invariants

- Appointment dates cannot be in the past; appointments are one hour long and start on 15-minute boundaries.
- A booking must fit entirely inside the latest effective working shift for the doctor and date.
- A booking cannot overlap doctor unavailability or another active future appointment.
- Only SCHEDULED appointments may be checked in or marked no-show; only CHECKED_IN appointments may be completed.
- Only active appointments may use active rescheduling; system-cancelled appointments use a separate replacement operation.

Billing invariants

- Only a DRAFT invoice may enter PAYMENT_PENDING; a successful payment must match the locked amount.
- Only a PAID invoice may enter REFUND_PENDING; a confirmed refund must match the approved amount.
- A refund ledger must reference a successful payment deduction.
- Money cannot be negative and arithmetic requires the same currency.
- Pricing applies discount evaluation followed by a fixed 10% tax rule.

2. Requirements and Architectural Drivers

2.1 Functional Requirements

ID	Requirement	System statement	Status
FR-01	Calendar configuration	The system shall maintain effective weekly schedules and temporary unavailability for each doctor.	Implemented
FR-02	Appointment booking	The system shall accept a one-hour appointment only when calendar, time-boundary, and overlap rules are satisfied.	Implemented
FR-03	Appointment lifecycle	The system shall support check-in, completion, no-show, and cancellation through state-protected operations.	Implemented
FR-04	Rescheduling	The system shall replace active or system-cancelled appointments while preserving domain history.	Partial: replacement event publication issue
FR-05	Proforma issuance	Billing shall create a draft invoice after consuming AppointmentWasBooked.	Implemented
FR-06	Pricing	Billing shall calculate a price from doctor profile, appointment duration, discount rules, and tax.	Implemented
FR-07	Checkout	Billing shall lock the invoice amount and create a Stripe PaymentIntent.	Implemented
FR-08	Financial finalization	Payment and refund states shall be finalized only from verified Stripe webhook results.	Implemented
FR-09	Cancellation and refund	Billing shall cancel an unpaid invoice or evaluate a paid invoice for refund.	Partial: PAYMENT_PENDING policy undefined
FR-10	Invoice expiration	Billing shall expire overdue invoices and notify Scheduling to cancel the appointment.	Implemented
FR-11	Queries	Each service shall expose dedicated read projections for appointments, invoices, and payments.	Implemented
FR-12	Pricing profiles	Billing shall create and update doctor pricing profiles.	Implemented

2.2 Non-Functional Requirements and ASRs

Architecturally Significant Requirements (ASRs) are the requirements that materially shape service boundaries, interaction style, data ownership, internal architecture, security, and deployment. The following table links each quality requirement to its architectural consequence.

ID	Quality attribute	Requirement	ASR	Evidence / status
NFR-01	Availability / temporal decoupling	Scheduling commands shall not synchronously depend on Billing availability.	ASR-01	RabbitMQ integration; partial reliability because no outbox
NFR-02	Modifiability	Scheduling rules, refund rules, and Stripe integration shall be changeable with limited cross-context impact.	ASR-02	Bounded contexts, policy objects, ports and adapters
NFR-03	Data ownership	No service shall read or write another service database.	ASR-03	Separate scheduling_db and billing_db
NFR-04	Security	Financial callbacks shall be accepted only after Stripe signature verification.	ASR-04	StripeSignatureVerifier and endpoint secret
NFR-05	Testability	Core business rules shall be executable without Spring, PostgreSQL, RabbitMQ, or Stripe.	ASR-05	Domain-centric unit tests and dependency inversion
NFR-06	Deployability	The complete prototype shall start reproducibly on a developer machine.	ASR-06	Dockerfiles and Docker Compose
NFR-07	Reliability / convergence	Cross-service workflows shall survive duplicate delivery and producer failure without permanent inconsistency.	ASR-07	Not satisfied: outbox, idempotency, retry, and DLQ absent
NFR-08	Observability	Operators shall be able to assess service health, event failures, and financial workflow latency.	ASR-08	Partially satisfied: infrastructure health checks only; no application metrics

ID	Quality attribute	Requirement	ASR	Evidence / status
NFR-09	Auditability	Financial actions shall be represented by persistent ledger records and provider transaction identifiers.	ASR-09	Payment aggregate and invoice history fields

2.3 Quality Attribute Scenarios

QAS-01 - Billing temporarily unavailable

Element	Scenario
Source	Billing Service failure
Stimulus	A valid appointment is booked while Billing is unavailable
Environment	Normal operation with RabbitMQ available
Artifact	Scheduling Service and appointment.exchange
Response	Scheduling commits the appointment and publishes a booking event without waiting for Billing; Billing processes the queued event after recovery
Measure	No synchronous Billing call on the booking path; eventual invoice creation after consumer recovery
Current status	Architectural intent implemented; guaranteed publication is weakened by the missing transactional outbox

QAS-02 - Refund rule changes

Element	Scenario
Source	Clinic management
Stimulus	The patient cancellation notice threshold changes
Environment	Maintenance release
Artifact	Billing refund policy package
Response	Eligibility and amount rules are changed inside Billing without modifying Scheduling or Stripe adapters
Measure	No Scheduling source or database schema change
Current status	Supported by policy objects and bounded-context ownership

QAS-03 - Forged webhook

Element	Scenario
Source	External attacker or malformed client
Stimulus	An unsigned or incorrectly signed payment callback is sent
Environment	Normal operation
Artifact	Billing Stripe webhook adapter
Response	The callback is rejected and no payment or invoice state is changed
Measure	Zero accepted callbacks with invalid signature
Current status	Signature verification is implemented; automated security test evidence should be added

QAS-04 - Duplicate event delivery

Element	Scenario
---------	----------

Element	Scenario
Source	RabbitMQ redelivery
Stimulus	AppointmentWasBooked or AppointmentWasCanceled is delivered more than once
Environment	Consumer restart or transient failure
Artifact	Billing event consumers and repositories
Response	The same business effect occurs at most once
Measure	One invoice and at most one refund per appointment/event identity
Current status	Required for production but not explicitly implemented; event IDs and an inbox/deduplication mechanism are recommended

2.4 Requirement-to-Architecture Traceability

Requirement / ASR	Architectural concern	Design and code evidence	Validation
FR-02 / ASR-05	Correct booking and testable domain rules	Appointment, AppointmentCalendar, AppointmentConflictDomainService	Domain unit tests
FR-05 / ASR-01	Invoice creation must not block booking	AppointmentWasBooked + RabbitMQ consumer	Manual event-driven demo; integration test recommended
FR-08 / ASR-04	Provider-confirmed financial state	Stripe webhook adapter + FinalizeTransactionService	Signed webhook demo; negative signature test recommended
FR-09 / ASR-02	Refund rules must be modifiable	Refund eligibility and amount policy packages	Billing policy unit tests
FR-10 / ASR-01	Unpaid reservation release	TimeOutTaskScheduler + InvoiceWasExpired	Demo with short timeout
NFR-03 / ASR-03	Independent data ownership	Repository adapters + two databases	Docker topology and source inspection
NFR-06 / ASR-06	Reproducible prototype deployment	Dockerfiles + docker-compose.yml	docker compose up --build
NFR-07 / ASR-07	Reliable eventual consistency	Outbox/inbox, retry, DLQ	Not implemented; documented architectural debt

2.5 Proposed Service Levels

The prototype does not currently export the metrics needed to claim measured service levels. The following SLOs are design targets for a production-oriented extension, not validated results:

Proposed SLO	SLI	Required instrumentation
SLO-01: 99% of valid appointment booking commands complete within 750 ms, excluding asynchronous invoice creation.	Histogram of successful POST /api/appointments latency.	Spring Boot Actuator, Micrometer timer, Prometheus scrape.
SLO-02: 99% of booking/cancellation events are handled within 5 seconds while broker and consumer are healthy.	Difference between event occurredOn and consumer completion timestamp.	Event ID, correlation ID, consumer timer, failure counter, dashboard.

3. Domain-Driven Design

3.1 Subdomains

Subdomain classification	Project responsibility
Core domain	Scheduling and the appointment-to-financial workflow: availability, appointment lifecycle, invoice state, cancellation, and refund decisions.
Supporting subdomains	Doctor calendar configuration, pricing profiles, discount composition, read projections, and timeout administration.
Generic/external capability	Payment execution through Stripe and message transport through RabbitMQ.

3.2 Bounded Contexts

Bounded context	Owned model and decisions	Database
Scheduling	Appointment, AppointmentCalendar, schedules, shifts, unavailability, conflicts, check-in, completion, no-show, cancellation, and rescheduling.	scheduling_db
Billing	AppointmentInvoice, Payment, PricingProfile, pricing/refund policies, Stripe transactions, reconciliation, and timeout processing.	billing_db

3.3 Context Map and Published Language

Scheduling is the source of operational appointment facts. Billing consumes those facts and is the source of invoice-expiration facts. The contexts cooperate through a Published Language consisting of integration-event contracts. No aggregate or JPA entity crosses the boundary.

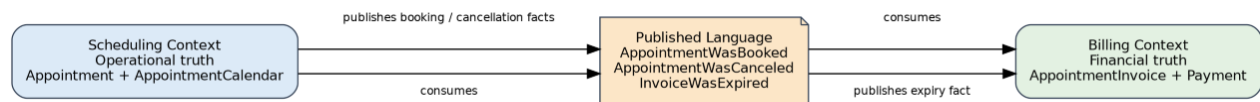


Figure 2. Context map and Published Language between Scheduling and Billing.

Event	Producer	Consumer	Purpose
AppointmentWasBooked	Scheduling	Billing	Create a draft invoice.
AppointmentWasCanceled	Scheduling	Billing	Cancel a draft or evaluate a paid invoice for refund.
InvoiceWasExpired	Billing	Scheduling	Cancel the unpaid appointment by system automation.

3.4 Aggregate Design

Appointment and AppointmentCalendar

Appointment owns identity, doctor, patient, a one-hour time slot, status, cancellation initiator, timestamps, and rescheduling lineage. Its methods guard lifecycle transitions and create integration events only for facts that must leave Scheduling. AppointmentCalendar selects the effective schedule, validates working time, checks unavailability, and collaborates with AppointmentConflictDomainService for rules involving multiple bookings.

AppointmentInvoice and Payment

AppointmentInvoice is the financial process aggregate for one appointment. It owns the invoice state machine, calculated amounts, approved refund amount, expiration deadline, and checkout window. Payment is a ledger

aggregate for a deduction or refund, with provider transaction identifiers and PENDING, SUCCESS, FAILED, or EXPIRED outcomes.

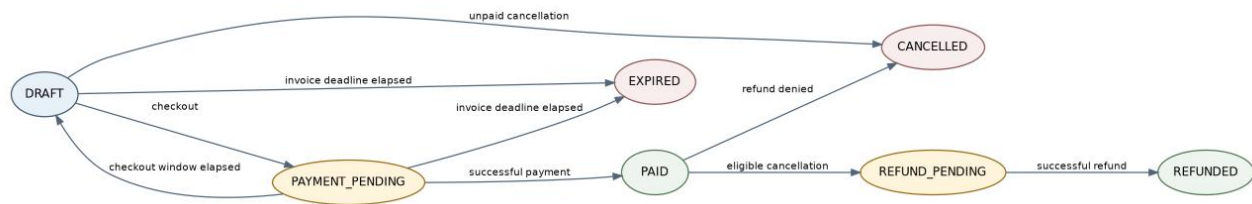


Figure 3. Implemented AppointmentInvoice state transitions.

3.5 Domain Policies

Billing separates decisions from orchestration. Pricing rules compare exclusive and stackable discounts before applying tax. Refund rules first decide eligibility and then select the amount strategy. This structure supports NFR-02: policy changes are localized and independently testable.

Cancellation case	Eligibility	Amount
Clinic reception or system automation	Force approve	Full refund
Patient within 15 minutes of booking	Force approve	Full refund
Patient with less than 24 hours notice	Deny	No refund
Patient with 24–48 hours notice	Approve	Paid total minus 20 currency units
Patient with more than 48 hours notice	Approve	Full refund

3.6 Business Process Flows

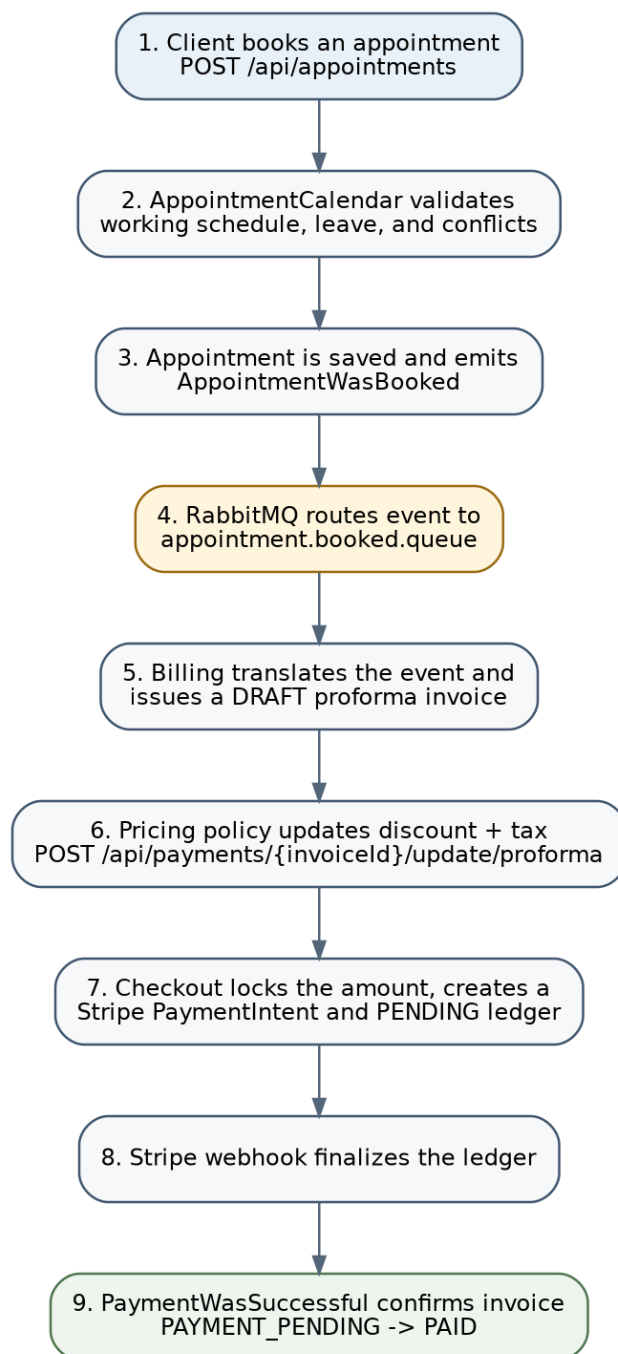


Figure 4. Booking, invoice creation, checkout, and payment confirmation.

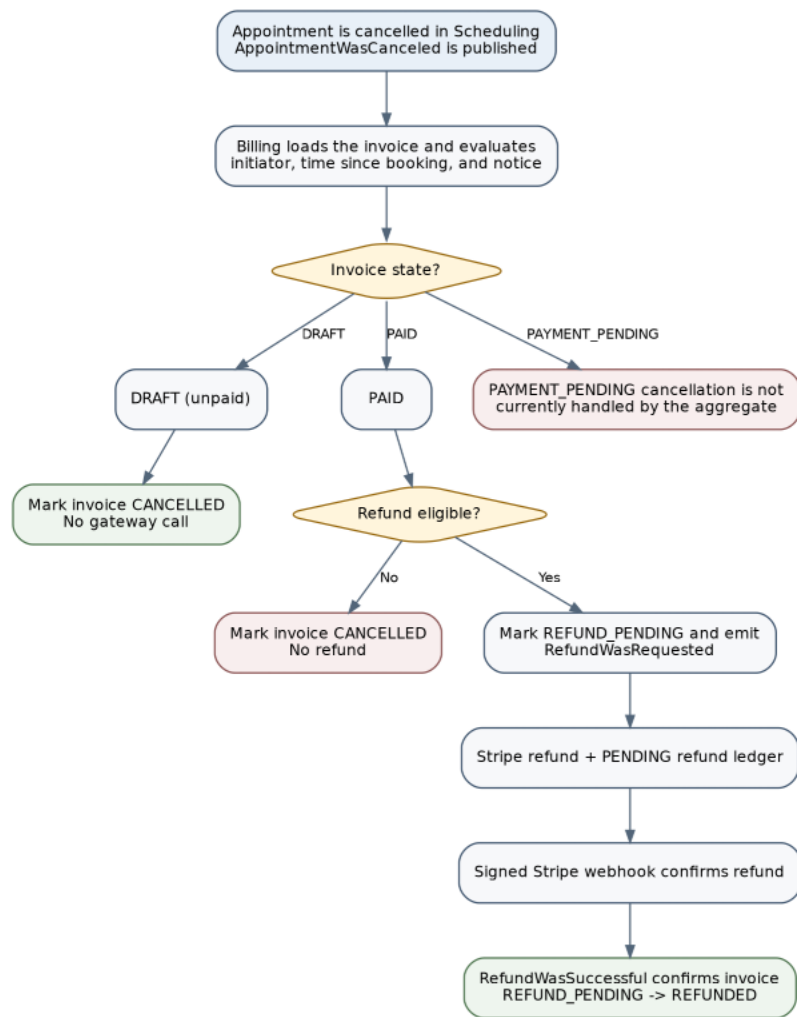


Figure 5. Unpaid cancellation and paid cancellation/refund paths.

Current process boundary

AppointmentInvoice.processCancellation explicitly supports DRAFT and PAID paths. A cancellation received while the invoice is PAYMENT_PENDING raises an invoice-state exception. This is an identified policy gap, not an implemented capability.

4. Software Architecture Views

4.1 Component-and-Connector View

This runtime view shows independently executing components and their connectors. HTTP/JSON is used for external commands and queries. AMQP is used between bounded contexts. JDBC connects each service only to its owned database. Stripe is a synchronous external API for operation initiation and an asynchronous signed-webhook source for final results.

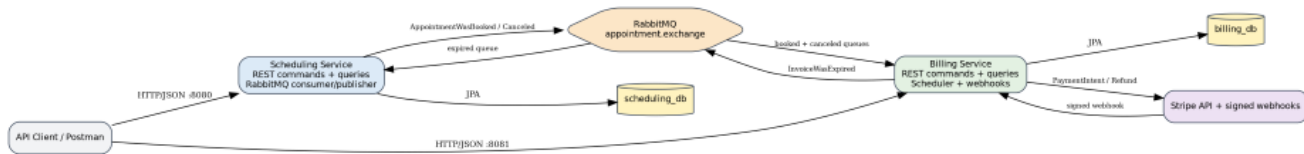


Figure 6. Component-and-Connector view.

4.2 Module View

Each service is partitioned by dependency direction. Domain modules contain aggregates, value objects, domain services, policies, and events. Application modules orchestrate use cases and define ports. Infrastructure modules provide REST, RabbitMQ, JPA, scheduling, configuration, and Stripe adapters. Dependencies point toward the domain; the two services have no compile-time dependency on each other.

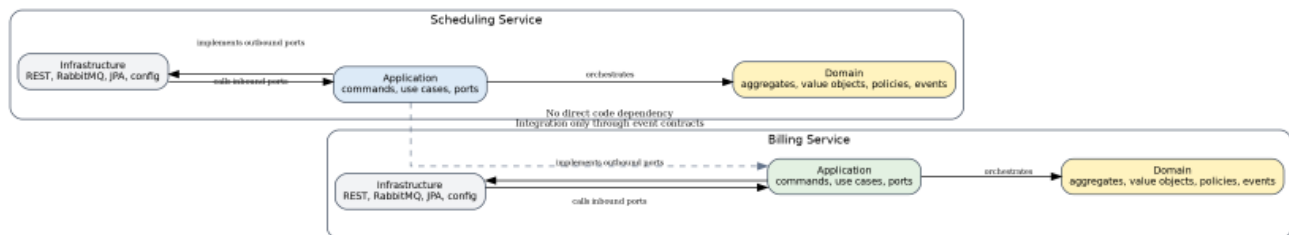


Figure 7. Module view and Hexagonal Architecture dependency direction.

4.3 Allocation / Deployment View

The prototype is allocated to a Docker Compose host. Scheduling and Billing run in separate containers, RabbitMQ provides the broker, PostgreSQL hosts two logical databases, and the Stripe CLI forwards test webhooks to Billing. Docker-internal DNS resolves service names.

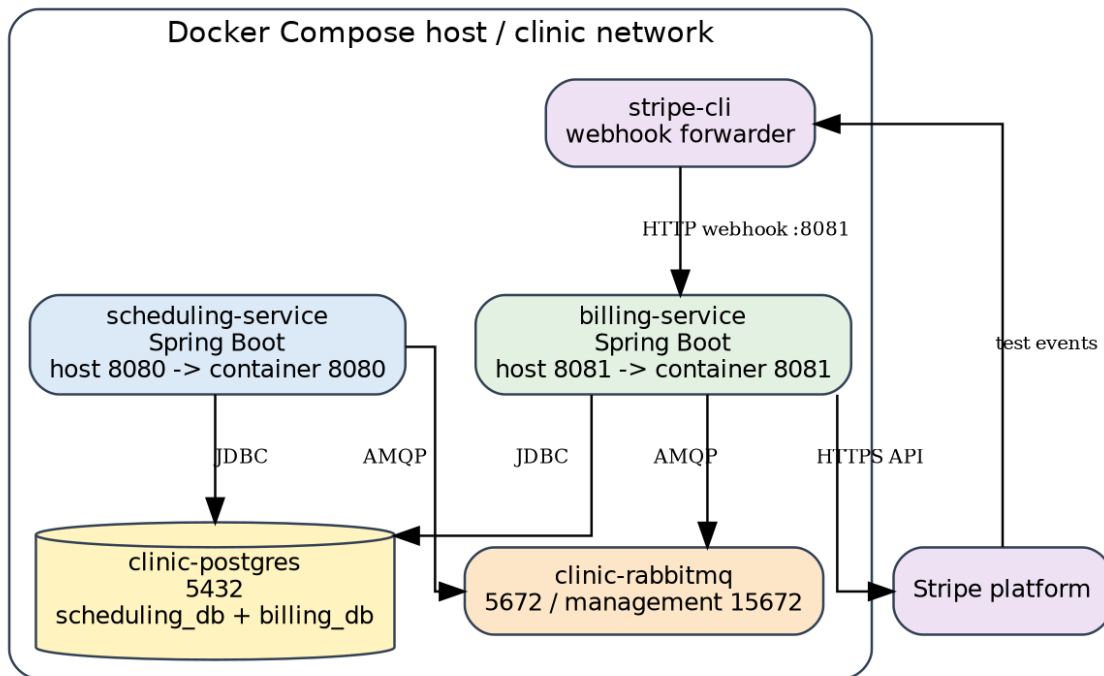


Figure 8. Allocation and deployment view.

4.4 Architectural Coupling and Cohesion

Property	Assessment
Functional cohesion	High inside each context: Scheduling groups temporal/appointment behavior; Billing groups financial behavior.
Static coupling	Low across services: no shared code module or cross-service repository access.
Dynamic coupling	Event schemas, routing keys, RabbitMQ availability, and eventual-consistency expectations couple the services at runtime.
Temporal coupling	Reduced because producers do not wait for consumers; retained messages allow delayed processing while queues remain available.
Architecture quantum	Each service plus its owned database forms a deployable quantum; RabbitMQ event contracts connect the quanta.
Main reliability risk	The database write and event publication are separate operations, and consumers are not explicitly idempotent.

5. Architectural Patterns and Decisions

5.1 Hexagonal Architecture

Business behavior is reached through inbound use-case interfaces and depends on outbound ports rather than concrete infrastructure. Examples include SchedulingUseCase and CalendarUseCase on the inbound side, AppointmentRepositoryPort and DomainEventPublisher on the outbound side, and PaymentGatewayPort and RefundGatewayPort around Stripe. REST controllers, RabbitMQ listeners, JPA adapters, scheduled tasks, and Stripe services remain replaceable adapters.

Role	Examples
Inbound ports	SchedulingUseCase, CalendarUseCase, ProformaUseCase, InitiateCheckoutUseCase, InitiateCancellationUseCase, FinalizeTransactionUseCase
Inbound adapters	REST controllers, RabbitMQ listeners, Stripe webhook controller, TimeOutTaskScheduler
Outbound ports	AppointmentRepositoryPort, InvoiceRepositoryPort, PaymentRepositoryPort, PaymentGatewayPort, RefundGatewayPort, DomainEventPublisher
Outbound adapters	JPA repository adapters, RabbitMQ publishers, ProcessPaymentService, RefundProcessService

5.2 Microservices and Database per Service

The service split follows bounded contexts rather than technical layers. Scheduling and Billing can be deployed independently and own separate schemas. This increases operational complexity but provides a concrete environment for studying distributed consistency, event contracts, failure modes, and independent evolution.

5.3 Asynchronous Messaging and RabbitMQ Decision

RabbitMQ was selected over synchronous REST because appointment commands should not wait for Billing and because topic routing and work-queue delivery fit the small set of business events. RabbitMQ was selected over Kafka because the prototype does not require retained event streams, consumer replay, large-scale stream processing, or Event Sourcing. Kafka remains a valid alternative if those requirements emerge. [SAP-L10, SAP-A3]

Reliability consequence

Asynchronous messaging removes synchronous availability dependence but does not automatically guarantee consistency. A transactional outbox, idempotent inbox, publisher confirms, retry rules, and dead-letter queues are required to harden the current design.

5.4 CQRS and Event-Driven Choreography

Command controllers invoke use cases and aggregates; query controllers use AppointmentQueryDao, InvoiceQueryDao, and PaymentQueryDao to return projections. This is lightweight CQRS without Event Sourcing. The cancellation/refund/expiry process is choreography-like because services react to events, but it is not claimed as a formal Saga: no explicit saga state, compensating-command model, or coordinator is implemented.

5.5 Pattern Selection and Scope

Pattern / technology	Decision	Rationale
Domain-Driven Design	Adopted	Ubiquitous language, aggregates, value objects, policies, bounded contexts, domain events.
Hexagonal / Clean Architecture	Adopted	Application ports isolate JPA, RabbitMQ, REST, and Stripe adapters.

Pattern / technology	Decision	Rationale
Microservices	Adopted	Scheduling and Billing are separately deployable and own separate databases.
Database per service	Adopted	scheduling db and billing db.
Asynchronous messaging	Adopted	RabbitMQ topic exchange and dedicated queues.
CQRS	Lightweight form	Command paths use aggregates; query DAOs return projections.
Saga	Not formally implemented	The cancellation/refund flow resembles choreography, but there is no explicit saga state or coordinator.
API Gateway	Excluded	Two APIs are accessed directly; centralized routing/authentication is outside prototype scope.
Service discovery	Excluded	Docker Compose DNS is sufficient for the local deployment.
Event Sourcing	Excluded	Current state is persisted conventionally; domain events are integration signals, not the source of truth.
Kafka	Excluded by decision	RabbitMQ fits routed work queues and short-lived business-event delivery; retained replayable streams are not required.
Kubernetes	Excluded	Docker Compose satisfies the prototype deployment and demonstration scope.
Observability stack	Planned	No Actuator/Micrometer/Prometheus/Grafana stack is currently present.
Reactive programming	Not adopted	The system is message-driven between services but uses imperative Spring MVC/JPA internally.

5.6 Architecture Decision Records

ADR	Decision	Status	Rationale
ADR-001	Separate Scheduling and Billing bounded contexts	Accepted	Different business language, rules, change drivers, and data ownership justify independent services.
ADR-002	Use asynchronous cross-context communication	Accepted	Scheduling must not wait for Billing; events expose eventual consistency.
ADR-003	Use RabbitMQ instead of Kafka	Accepted	Topic routing and work-queue semantics fit the prototype; replayable event streams are not required.
ADR-004	Use database per service	Accepted	Prevents cross-context joins and preserves ownership.
ADR-005	Use Hexagonal Architecture inside each service	Accepted	Keeps domain logic independent from Spring, JPA, RabbitMQ, and Stripe.
ADR-006	Use lightweight CQRS	Accepted	Write models protect invariants; read models provide direct projections without Event Sourcing.
ADR-007	Deploy with Docker Compose	Accepted for prototype	Provides a reproducible local environment; Kubernetes is not justified for the current scope.
ADR-008	Do not introduce an API Gateway	Accepted for prototype	The system has only two directly exposed APIs and no centralized authentication requirement in scope.

5.7 Architecture Governance and Fitness Functions

The repository does not currently automate architecture governance. The following fitness functions are suitable additions:

- ArchUnit: domain packages must not depend on infrastructure packages or Spring/JPA/Stripe types.
- ArchUnit: controllers may depend on inbound ports but not directly on JPA repositories.
- Contract test: every published integration event must include a stable event identity, occurrence time, and version.
- Build check: service Dockerfiles and Compose port mappings must agree.

- Integration check: duplicate event delivery must not create a second invoice or refund.

6. Prototype Implementation

6.1 Technology Stack

Area	Implementation
Language	Java 17
Application framework	Spring Boot 4.0.6 in Scheduling; 4.1.0 in Billing
API	Spring Web MVC and Jakarta Validation
Persistence	Spring Data JPA / Hibernate and PostgreSQL
Messaging	Spring AMQP and RabbitMQ topic exchange
Payments	Stripe Java SDK 33.1.0 and Stripe CLI
Build and packaging	Maven Wrapper and multi-stage Dockerfiles
Deployment	Docker Compose with RabbitMQ 3 Management and PostgreSQL 15 Alpine

6.2 Application Services

Service / use case	Responsibility
SchedulingService	Validate availability, change Appointment, persist, and publish booking/cancellation facts.
CalendarService	Change schedules/unavailability, detect conflicts, cancel affected appointments, and publish events.
ProformaService	Create invoice from booking event and recalculate price before checkout.
InitiateCheckoutService	Lock amount, call PaymentGatewayPort, and store pending deduction ledger.
FinalizeTransactionService	Apply verified Stripe results to ledger and invoice.
InitiateCancellationService	Build refund context, evaluate invoice cancellation, and publish internal refund request.
RefundInitiationService	Call Stripe refund adapter and create pending refund ledger.
TimeOutService	Release checkout windows or expire invoices and publish InvoiceWasExpired.

6.3 Stripe Integration Boundary

- ProcessPaymentService converts domain Money to Stripe minor units and returns a provider-neutral PaymentIntentResponse.
- RefundProcessService creates a refund against the original PaymentIntent identifier.
- StripeSignatureVerifier validates the webhook payload and endpoint secret.
- Translators convert provider events into application commands; unrelated event types are ignored.
- Invoices are not marked PAID or REFUNDED when the outbound API call is made; state changes follow the verified webhook.

6.4 Persistence and Read Models

Domain objects are mapped to JPA entities by explicit mappers. Repository adapters translate data-access failures into application-specific exceptions. Aggregate repositories rebuild write models, while query DAOs return API projections directly. The prototype uses Hibernate ddl-auto=update; migration tooling would be preferable for controlled production evolution.

6.5 Public HTTP Interface

Service	Method	Endpoint	Purpose
Scheduling	POST	/api/appointments	Book appointment
Scheduling	POST	/api/appointments/{id}/cancel	Cancel appointment
Scheduling	POST	/api/appointments/{id}/checkin	Check in
Scheduling	POST	/api/appointments/{id}/complete	Complete appointment
Scheduling	POST	/api/appointments/{id}/noshow	Mark no-show
Scheduling	POST	/api/appointments/{id}/reschedule/active	Reschedule active appointment
Scheduling	POST	/api/appointments/{id}/reschedule/system-canceled	Replace system-cancelled appointment
Scheduling	POST	/api/calendars/{doctorId}/unavailabilities	Add unavailability
Scheduling	POST	/api/calendars/{doctorId}/schedules	Add effective schedule
Scheduling	GET	/api/queries/appointments/...	Appointment projections
Billing	POST	/api/profile/create	Create pricing profile
Billing	POST	/api/profile/update	Intended profile update; currently wired to create
Billing	POST	/api/payments/{invoiceId}/update/proforma	Recalculate invoice
Billing	POST	/api/payments/{invoiceId}/initiate/payment	Create PaymentIntent
Billing	POST	/api/stripe/webhook/payments	Receive signed Stripe events
Billing	GET	/api/queries/invoices/...	Invoice projections
Billing	GET	/api/queries/payments/...	Payment projections

REST qualification

The prototype exposes HTTP/JSON command and query APIs. Several workflow operations use explicit command endpoints such as /initiate/payment and /reschedule/active, so the report does not claim full resource-oriented REST maturity.

7. Validation Strategy

7.1 Testing Pyramid

The course distinguishes unit, integration, component, and end-to-end tests. The project is strongest at the domain-unit layer. The remaining layers should be presented as a documented validation plan unless executable evidence is added. [SAP-L9, SAP-A2]

Level	Scope	Current evidence	Assessment
Unit - domain	Aggregates, value objects, pricing rules, refund rules, time overlap and state transitions.	11 Scheduling test files and 29 Billing test files are present. Older Scheduling Surefire artifacts record 54 passing tests.	Strongest current layer
Unit - application	Use cases with mocked repositories, gateways, publishers, and clocks.	Some application-oriented tests exist, but coverage is not reported.	Expand and record results
Integration	JPA mappings, PostgreSQL repositories, RabbitMQ contracts, Stripe signature translation.	No complete automated suite is documented.	Required improvement
Component	Each service tested through its API with external dependencies stubbed or containerized.	Postman/manual flows can support this, but automated evidence is absent.	Required improvement
End-to-end	Booking -> invoice -> Stripe payment -> webhook -> cancellation/refund/expiry.	Docker Compose demonstration is defined; no automated E2E suite is present.	Manual validation only

7.2 Current Evidence

- The source snapshot contains 11 Scheduling test files and 29 Billing test files.
- Older Scheduling Surefire reports included in the archive record 54 passing tests across Appointment, TimePeriod, WeeklySchedule, and WorkingShift.
- Those reports do not prove a clean build of the current complete source snapshot.
- Before submission, run both Maven suites from a clean checkout and record totals, failures, Java version, and execution date in docs/validation.md.

7.3 Required Integration and End-to-End Checks

Validation	Expected result
Booking event contract	One valid booking creates one draft invoice with matching appointment and doctor IDs.
Duplicate booking event	A redelivery does not create a second invoice after idempotency is implemented.
Invalid Stripe signature	Webhook returns an error and changes no ledger or invoice state.
Payment success	Verified webhook moves pending deduction to SUCCESS and invoice to PAID.
Paid cancellation	Policy result determines denial, penalty refund, or full refund; provider callback finalizes the invoice.
Invoice expiry	Short demo timeout publishes InvoiceWasExpired and Scheduling cancels the appointment.
Consumer outage	Queued events are processed after Billing restarts.

7.4 Demonstration as Validation

The oral presentation can treat the live demonstration as a controlled end-to-end validation. The presenter should state the expected state before each step, execute the operation, query both contexts, and explain any temporary inconsistency caused by asynchronous delivery.

1. Create a doctor pricing profile in Billing.

2. Create an effective weekly schedule in Scheduling.
3. Book a valid appointment and show the immediate Scheduling result.
4. Query Billing after event delivery and show the DRAFT invoice.
5. Recalculate the proforma and initiate checkout.
6. Complete a Stripe test payment and show PAID only after the webhook.
7. Cancel the appointment and explain refund eligibility and amount.
8. Run a second unpaid scenario and show automatic expiry and system cancellation.

7.5 Observability Validation Gap

RabbitMQ and PostgreSQL have Docker health checks, but the Spring services do not expose application health or metrics. Course-aligned observability would add Actuator health endpoints, Docker health checks for both services, Micrometer counters/timers, Prometheus, and a small Grafana dashboard. [SAP-L8, SAP-A2]

Suggested metric	Purpose
appointments_booked_total	Booking volume
appointment_booking_failures_total	Rejected or failed booking commands
invoice_creation_latency_seconds	Event propagation and Billing processing
payment_success_total / payment_failure_total	Financial outcome
refund_success_total / refund_failure_total	Refund outcome
event_consumer_failures_total	Messaging reliability
invoice_expirations_total	Timeout behavior

8. Deployment and Demonstration

8.1 Container Topology

Container	Host ports	Purpose
clinic-rabbitmq	5672, 15672	AMQP broker and management UI; health checked.
clinic-postgres	5432	PostgreSQL server hosting scheduling_db and billing_db; persistent volume.
scheduling-service	8080	Scheduling command/query API and InvoiceWasExpired consumer.
billing-service	8081	Billing API, event consumers, timeout scheduler, and Stripe webhook.
stripe-cli	none	Forwards Stripe test events to Billing inside the Compose network.

8.2 Environment and Secrets

The root environment file must provide STRIPE_API_KEY and STRIPE_PAYMENT_WEBHOOK_SECRET. The uploaded archive contains .env files; these must not be committed to GitHub or included in the submission. Any real key already shared should be rotated. Keep only .env.example files with placeholders.

8.3 Running the Prototype

`docker compose up --build`

`docker compose ps`

`docker compose logs -f scheduling-service billing-service stripe-cli`

`docker compose down`

`docker compose down -v`

8.4 Stripe Webhook Setup

1. Start RabbitMQ, PostgreSQL, Scheduling, and Billing.
2. Start stripe-cli and read the whsec_... secret printed for the active listener.
3. Place that value in STRIPE_PAYMENT_WEBHOOK_SECRET.
4. Recreate Billing so the endpoint secret is reloaded.
5. Trigger a Stripe test event and verify a successful signed callback.

8.5 Production and Demo Timeout Profiles

The checked-in Billing property clinic.checkout.payment-timeout is currently 20 seconds, and the timeout scheduler runs every five seconds. This is a demonstration-oriented value. A production narrative of two hours and a demo override of 15 seconds becomes accurate only after explicit profile or Compose configuration is added.

Profile	Configuration	Purpose
Current source	clinic.checkout.payment-timeout=20s	Fast local demonstration.
Suggested demo	CLINIC_CHECKOUT_PAYMENT_TIMEOUT=15s	Observe timeout behavior immediately.

Profile	Configuration	Purpose
Suggested production-like	CLINIC_CHECKOUT_PAYMENT_TIMEOUT=2h	Realistic checkout grace period.

9. Trade-Offs, Limitations, and Roadmap

9.1 Architectural Trade-Offs

Decision	Benefit	Cost / consequence
Two bounded-context microservices	Independent models, data ownership, and deployability.	More deployment, messaging, failure, and consistency complexity than a modular monolith.
Asynchronous events	Temporal decoupling and non-blocking cross-context workflows.	Eventual consistency, duplicate delivery, ordering, and transaction/publication risks.
Hexagonal Architecture	Testable domain and replaceable adapters.	More interfaces, mapping code, and package structure.
Stripe webhook finalization	Provider-confirmed financial truth.	Delayed outcomes and need for signature security and reconciliation.
Lightweight CQRS	Clear write invariants and efficient projections.	Two paths to maintain; no historical reconstruction as with Event Sourcing.

9.2 Prioritized Constraints

Priority	Area	Current condition	Action
High	Messaging atomicity	Database changes and RabbitMQ publication are not atomic.	Transactional outbox, publisher confirms, and relay worker.
High	Duplicate/error handling	No event ID, inbox/deduplication store, bounded retry, or DLQ.	Add idempotent consumers, retry policy, and dead-letter routing.
High	Reschedule replacement event	System-cancelled replacement creates a new booking event but publishes from the old appointment.	Publish and clear events from the new appointment.
Medium	Pending payment cancellation	AppointmentInvoice has no explicit PAYMENT PENDING cancellation policy.	Cancel PaymentIntent, defer resolution, or expire the ledger through a defined policy.
Medium	Optimistic locking	Scheduling catches optimistic-lock exceptions but AppointmentEntity has no @Version.	Add versioning or remove unsupported handling.
Medium	Profile update endpoint	/api/profile/update invokes createProfile.	Translate to UpdateProfileCommand and invoke update use case.
Medium	Observability	Application health, metrics, tracing, and alerting are absent.	Add Actuator, Micrometer, Prometheus, and correlation IDs.
Low	Billing Dockerfile metadata	Application listens on 8081 while Dockerfile exposes 8080.	Change EXPOSE to 8081.
Low	Demo configuration	Checked-in payment timeout is 20 seconds.	Use explicit production and demo profiles; document the effective value.

10. Conclusion

The Smart Clinic System is a credible SAP course project because the architecture follows the business problem. DDD provides a shared language and meaningful bounded contexts. Aggregates and policies protect non-trivial scheduling and financial rules. Hexagonal Architecture isolates infrastructure. RabbitMQ makes cross-context workflows asynchronous and exposes eventual consistency. Docker Compose makes the prototype reproducible.

The strongest academic position is not that every microservice pattern has been implemented. It is that the selected patterns are justified by architectural drivers, their consequences are understood, excluded patterns are

explicitly scoped out, and validation gaps are reported honestly. The current prototype demonstrates the architecture; the next stage is reliability, observability, and test automation rather than additional business breadth.

Appendix A. Course Alignment Matrix

SAP topic	Project status	Evidence / position
Analysis to deployment	Covered	Requirements, domain analysis, architecture views, implementation mapping, validation, Docker deployment.
Functional/non-functional requirements	Covered	FR/NFR catalogue and traceability.
Architectural drivers and ASRs	Covered	ASR list and Quality Attribute Scenarios.
C&C, module, allocation structures	Covered	Three explicit architecture views.
DDD and analysis practices	Covered	Ubiquitous language, user stories, Event Storming-style flow, bounded contexts, context map.
Hexagonal/Clean Architecture	Implemented	Ports and adapters in both services.
Microservices and interaction styles	Implemented	Two services, database per service, asynchronous events.
CQRS	Partially implemented	Separate command and query paths; no Event Sourcing.
Saga	Discussed, not formalized	Choreography-like workflow without saga coordinator/state.
Observability patterns	Gap documented	Infrastructure health checks only; application health/metrics planned.
Testing pyramid	Partially covered	Strong unit layer; integration/component/E2E automation remains.
Event-driven architecture	Implemented with RabbitMQ	Kafka and AsyncAPI are not used; choice is justified.
SLO/SLI	Proposed only	Two future service-level objectives defined; no measurement instrumentation.
Kubernetes	Excluded by scope	Docker Compose is used for prototype deployment.
ADRs and governance	Covered in documentation	ADR set and proposed architecture fitness functions.

Appendix B. Source-Code Evidence Map

Report topic	Primary location
Scheduling aggregate	scheduling-service/.../domain/Appointment.java
Calendar aggregate	scheduling-service/.../domain/AppointmentCalendar.java
Conflict domain service	scheduling-service/.../domain/service/AppointmentConflictDomainService.java
Scheduling use cases	scheduling-service/.../application/service/SchedulingService.java and CalendarService.java
Scheduling events	scheduling-service/.../domain/events/AppointmentWasBooked.java and AppointmentWasCanceled.java
Invoice aggregate	billing-service/.../domain/invoice/AppointmentInvoice.java
Payment aggregate	billing-service/.../domain/payment/Payment.java
Pricing policies	billing-service/.../domain/policy/pricing/...
Refund policies	billing-service/.../domain/policy/refund/...
Stripe adapters	billing-service/.../infrastructure/gateway/stripe/...
RabbitMQ integration	both services: infrastructure messaging/config packages
Deployment	root docker-compose.yml, service Dockerfiles, postgres-init/init.sql

Appendix C. Public API Summary

Service	Method	Endpoint	Purpose
Scheduling	POST	/api/appointments	Book appointment
Scheduling	POST	/api/appointments/{id}/cancel	Cancel appointment
Scheduling	POST	/api/appointments/{id}/checkin	Check in
Scheduling	POST	/api/appointments/{id}/complete	Complete appointment
Scheduling	POST	/api/appointments/{id}/noshow	Mark no-show
Scheduling	POST	/api/appointments/{id}/reschedule/active	Reschedule active appointment
Scheduling	POST	/api/appointments/{id}/reschedule/system-canceled	Replace system-cancelled appointment
Scheduling	POST	/api/calendars/{doctorId}/unavailabilities	Add unavailability
Scheduling	POST	/api/calendars/{doctorId}/schedules	Add effective schedule
Scheduling	GET	/api/queries/appointments/...	Appointment projections
Billing	POST	/api/profile/create	Create pricing profile
Billing	POST	/api/profile/update	Intended profile update; currently wired to create
Billing	POST	/api/payments/{invoiceId}/update/proforma	Recalculate invoice
Billing	POST	/api/payments/{invoiceId}/initiate/payment	Create PaymentIntent
Billing	POST	/api/stripe/webhook/payments	Receive signed Stripe events
Billing	GET	/api/queries/invoices/...	Invoice projections
Billing	GET	/api/queries/payments/...	Payment projections

Appendix D. Reference Material

[SAP-EXAM] SAP 2025/2026, About the Exam, <https://github.com/sap-2025-2026/Docs/blob/main/exam.md>

[SAP-QUESTIONS] SAP 2025/2026, Oral Exam: Example Questions, <https://github.com/sap-2025-2026/Docs/blob/main/questions.md>

[SAP-A1] SAP 2025/2026, Assignment 01, <https://github.com/sap-2025-2026/assignments/blob/main/assignment-01.md>

[SAP-A2] SAP 2025/2026, Assignment 02, <https://github.com/sap-2025-2026/assignments/blob/main/assignment-02.md>

[SAP-A3] SAP 2025/2026, Assignment 03, <https://github.com/sap-2025-2026/assignments/blob/main/assignment-03.md>

[SAP-L8] SAP 2025/2026, Lab Activity 08 - Observability Patterns, <https://github.com/sap-2025-2026/lab-activity-08>

[SAP-L9] SAP 2025/2026, Lab Activity 09 - Testing Microservices, <https://github.com/sap-2025-2026/lab-activity-09>

[SAP-L10] SAP 2025/2026, Lab Activity 10 - Event-Driven Architectures, <https://github.com/sap-2025-2026/lab-activity-10>